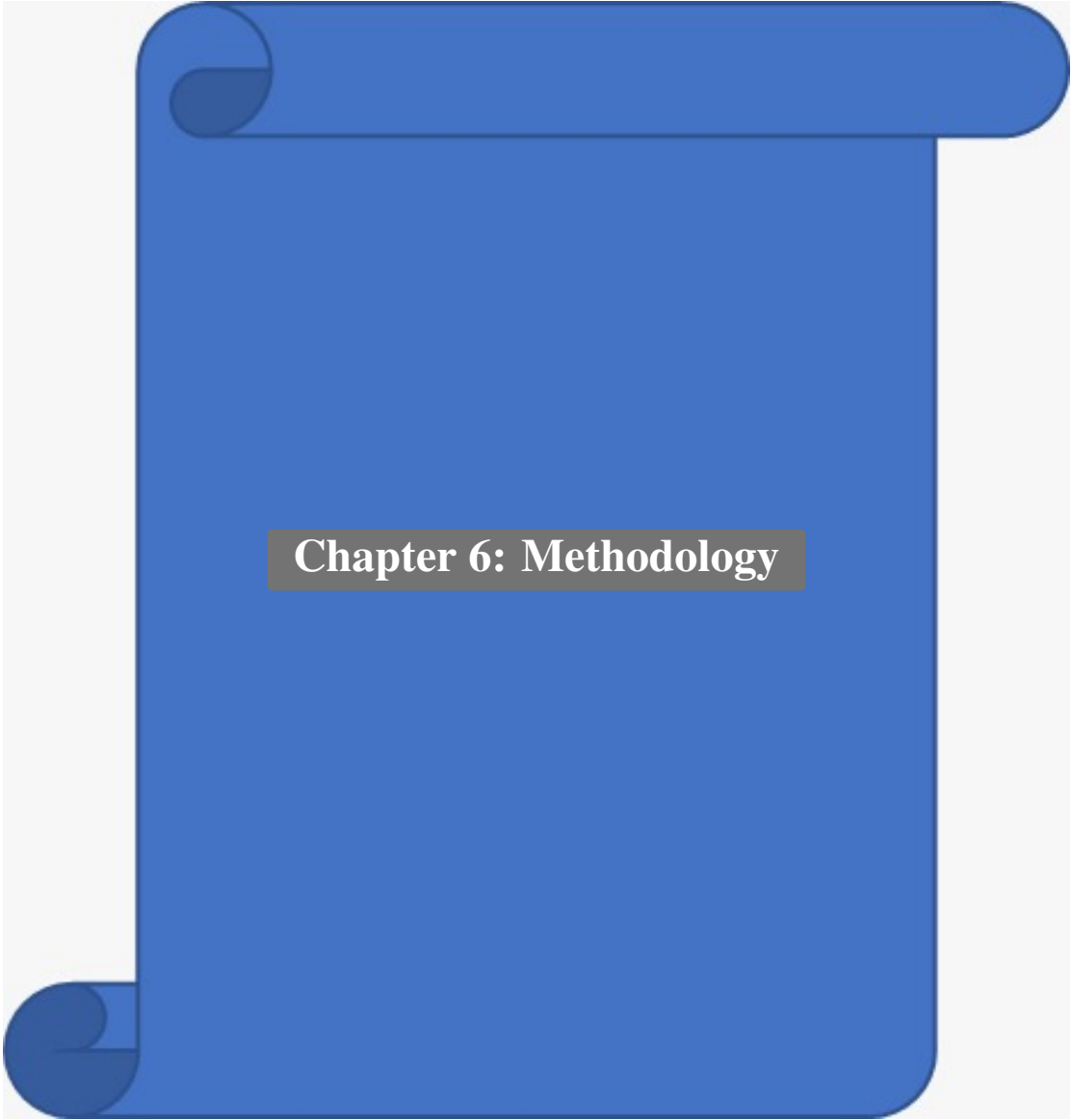




Chapter 6: Methodology

The development of AgriTrace followed an Agile methodology with iterative development cycles. The project was divided into sprints, each focusing on specific functional modules.

6.1 Development Approach

The Agile Scrum methodology was adopted for its flexibility and iterative nature. The development was organized into the following phases:

1. **Requirement Gathering:** Studied the agricultural waste management ecosystem, identified stakeholder needs (farmers, agents, administrators), and defined functional and non-functional requirements.
2. **System Design:** Designed the system architecture using a component-based approach with Next.js App Router. Created data models for Firestore collections and defined security rules for role-based access control.
3. **Frontend Development:** Built the user interface using React components with TypeScript, styled using Tailwind CSS and Radix UI. Implemented responsive layouts for all device sizes.
4. **Backend Integration:** Configured Firebase services (Firestore, Authentication, Storage), implemented API routes for payment processing, and integrated location and notification services.
5. **Feature Implementation:** Developed core features including waste reporting, listing marketplace, payment integration, carbon credit tracking, and the rewards system.
6. **Testing and Debugging:** Performed unit testing, integration testing, and user acceptance testing. Applied ESLint and TypeScript type-checking for code quality.
7. **Deployment:** Deployed the application to Firebase Hosting with CI/CD pipeline integration.

6.2 Sprint Breakdown

The development was organized into six two-week sprints. The following table summarizes the sprint plan:

6.3 Architecture Pattern

AgriTrace employs a **three-tier architecture**:

- **Presentation Layer:** React components with Next.js App Router, Tailwind CSS styling, and Radix UI primitives for accessible, composable UI elements.

Table 6.1: Sprint-wise Development Plan

Sprint	Focus Area	Deliverables	Duration
Sprint 1	Project Setup	Next.js project initialization, Firebase configuration, authentication setup, role-based routing	2 weeks
Sprint 2	Core Features	Waste listing CRUD operations, Firestore data models, listing status workflow (OPEN → ASSIGNED → IN_TRANSIT → DELIVERED)	2 weeks
Sprint 3	Dashboard UIs	Farmer dashboard, Agent dashboard, Admin dashboard with role-specific components and statistics cards	2 weeks
Sprint 4	Integrations	Razorpay payment gateway, Cloudinary photo uploads, GPS location services, notification system	2 weeks
Sprint 5	Advanced Features	Carbon credit tracking, rewards/gamification system, AI waste classification via Genkit, CSV export	2 weeks
Sprint 6	Polish & Deploy	Responsive design refinements, performance optimization, security rules, accessibility improvements, deployment	2 weeks

- **Application Layer:** Next.js API routes for server-side processing (Razorpay webhooks, admin operations), Firebase service functions for data operations, and custom React hooks for state management.
- **Data Layer:** Firebase Firestore for persistent NoSQL storage, Firebase Authentication for identity management, and Firebase Storage for media files (waste photos, delivery proofs).

6.4 Development Tools and Environment

The following tools were used throughout the development process:

6.5 Data Flow

The data flow in AgriTrace follows a unidirectional pattern:

1. Farmer creates a waste listing with details (type, quantity, location, photos).
2. Listing is stored in Firestore and becomes visible to agents.
3. Agent claims the listing and initiates pickup.
4. Agent marks the waste as “In Transit” and later as “Delivered”.

Table 6.2: Development Tools

Tool	Version	Purpose
Visual Studio Code	1.85+	Primary code editor with TypeScript IntelliSense
Node.js	18.x LTS	JavaScript runtime environment
npm	9.x	Package manager for dependencies
Git	2.40+	Distributed version control
GitHub	–	Repository hosting and collaboration
Firebase CLI	13.x	Firebase project management and deployment
ESLint	8.x	Static code analysis and linting
Prettier	3.x	Code formatting and style enforcement
Chrome DevTools	–	Browser debugging and performance profiling
Postman	10.x	API testing and documentation

5. Admin can mark listings as “Recycled” after processing.
6. Carbon credits are calculated based on waste type and quantity.
7. Payment is processed through Razorpay upon delivery confirmation.
8. Notifications are sent to relevant parties at each status change.